შესავალი QGIS-ის პროგრამირებაში

C++ PyQGIS-იდან QGIS Core-მდე

გიორგი კაპანაძე

თბილისი · 2026

სარჩევი

01 **რუკა: Python-იდან C++-მდე** 5

01.1 PyQGIS არის C++-ის სარკე
01.2 QGIS-ის არქიტექტურა
01.3 ერთი და იგივე ამოცანა ორ ენაზე
01.4 პირველი C++ პროგრამა
01.5 Python vs C++ ერთი შეხედვით
01.6 სავარჯიშო და შეჯამება

02 **ტიპები და ცვლადები** 10

02.1 ცვლადი როგორც ეტიკეტიანი ყუთი
02.2 ძირითადი ტიპები
02.3 კოდი შედეგით
02.4 const და auto
02.5 მთელი რიცხვების გაყოფა
02.6 სიზუსტე და გარდაქმნა
02.7 სავარჯიშო და შეჯამება

03 **ფუნქციები და namespace** 16

03.1 ფუნქციის ანატომია
03.2 namespace — სახელების საქაღალდე
03.3 overload — ერთი სახელი, სხვადასხვა პარამეტრები
03.4 default პარამეტრი
03.5 მანძილი ორ წერტილს შორის
03.6 .h და .cpp ფაილები
03.7 სავარჯიშო და შეჯამება

04 **მაჩვენებლები (Pointers)** 24

04.1 კურიერი და მისამართი
04.2 პირველი მაჩვენებელი
04.3 მაჩვენებელი ობიექტზე: ისარი
04.4 nullptr — „ფენა არ არის არჩეული“
04.5 ჩამოკიდებული მაჩვენებელი
04.6 სავარჯიშო და შეჯამება

05 მითითებები (References) და const 30

05.1 მეტსახელი
05.2 მაჩვენებელი თუ მითითება?
05.3 ფუნქციაში გადაცემა: კოპირება თუ ორიგინალი
05.4 const & — უსაფრთხო დათვალიერება
05.5 QGIS-ის ხელმოწერების კითხვა
05.6 სავარჯიშო და შეჯამება

06 კლასები და ობიექტები 37

06.1 კლასი და ობიექტი
06.2 public და private
06.3 სრული მაგალითი
06.4 კონსტრუქტორი
06.5 const მეთოდი
06.6 struct და class
06.7 სავარჯიშო და შეჯამება

07 მემკვიდრეობა და პოლიმორფიზმი 42

07.1 მემკვიდრეობა
07.2 virtual და override
07.3 ტიპის გარდაქმნა: dynamic_cast
07.4 qobject_cast QGIS-ში
07.5 სავარჯიშო და შეჯამება

08 მეხსიერების მართვა და smart pointers 48

08.1 stack და heap
08.2 new და delete
08.3 unique_ptr: ერთადერთი მფლობელი
08.4 მფლობელობის გადაცემა: std::move
08.5 shared_ptr: ბევრი მფლობელი
08.6 რა ხდება QGIS-ში
08.7 სავარჯიშო და შეჯამება

09 STL: კონტეინერები და ალგორითმები 55

09.1 vector — დინამიური მასივი
09.2 map — გასაღები → მნიშვნელობა
09.3 ალგორითმები
09.4 lambda — უსახელო ფუნქცია
09.5 ძებნა
09.6 სავარჯიშო და შეჯამება

10 Qt: QString, QVariant, signals და slots 62
10.1 QString
10.2 QList
10.3 QVariant — უნივერსალური ყუთი
10.4 QgsAttributes — ატრიბუტების ცხრილის ერთი სტრიქონი
10.5 QObject-ის ხე
10.6 signals და slots
10.7 სავარჯიშო და შეჯამება

11 api.qgis.org-ის კითხვა 68
11.1 კლასის გვერდის სქემა
11.2 ხელმოწერის გაშიფვრა
11.3 მაკროები, რომლებიც შეგხვდება
11.4 ცოცხალი მაგალითი: ამოცანიდან მეთოდამდე
11.5 ძირითადი კლასები ერთ ცხრილში
11.6 შეცდომის კითხვა
11.7 სავარჯიშო და შეჯამება

12 პროექტი: საკუთარი ინსტრუმენტი QGIS-ში 74
12.1 გეგმა
12.2 საფეხური 1: პირველი პროგრამა qgis_core-ზე
12.3 CMake
12.4 საფეხური 2: C++ პლაგინის ჩონჩხი
12.5 საფეხური 3: QGIS-ის აწყობა წყაროდან
12.6 სავარჯიშო და შეჯამება

CHAPTER 01

01

რუკა: Python-იდან C++-მდე

The map — from Python to C++

PyQGIS როგორც C++ API-ს სარკე · QGIS-ის არქიტექტურა · პირველი C++ პროგრამა

წინამდებარე სახელმძღვანელოს მიზანია, მკითხველს დაეხმაროს PyQGIS-ის საწყისი ცოდნიდან პროფესიულ დონეზე გადასვლაში, QGIS-ის C++ API-ის დოკუმენტაციის (api.qgis.org) სიღრმისეულად გააზრებასა და საკუთარი პროგრამული კოდის QGIS-ის ბირთვთან ინტეგრირების უნარის განვითარებაში.

გამოცემაში განხილული ყველა პრაქტიკული მაგალითი ეფუძნება გეოგრაფიულ მონაცემებს და მოიცავს ისეთ ფუნდამენტურ კომპონენტებს, როგორებიცაა სივრცითი ფენები, გეოგრაფიული ობიექტები, კოორდინატები და სივრცითი მახასიათებლები (მაგალითად, ფართობები).

01.1 PyQGIS არის C++-ის სარკე

PyQGIS ცალკე ბიბლიოთეკა არ არის. QGIS-ის ბირთვი C++-ზეა დაწერილი, Python-ის კლასები კი ავტომატურად გენერირდება (SIP-ის საშუალებით) იმავე C++ კლასებიდან. ამიტომ სახელები და ლოგიკა ერთი და იგივეა: `QgsVectorLayer`, `QgsFeature`, `QgsGeometry`.

რა არის SIP?

SIP (aPython–C++ Binding Generator) არის ინსტრუმენტი, რომელიც C++-ზე დაწერილი კლასებისა და ფუნქციების Python-იდან გამოყენების შესაძლებლობას უზრუნველყოფს. QGIS-ში SIP გამოიყენება C++ კლასებსა და Python ინტერფეისს შორის დამაკავშირებელი კოდის გენერირებისთვის.

QGIS-ის ბირთვი დაწერილია C++-ზე, ხოლო PyQGIS Python-იდან ამ ბირთვის ფუნქციონალზე წვდომის საშუალებას იძლევა. SIP-ის მეშვეობით იქმნება დამაკავშირებელი ფენა, რომლის საშუალებითაც Python-ის კოდიდან შესაძლებელია QGIS-ის C++ კლასებისა და მეთოდების გამოყენება.

მაგალითად, PyQGIS-ში შეგვიძლია გამოვიყენოთ შემდეგი კლასები:

- `QgsVectorLayer` — ვექტორული ფენის წარმოდგენა და მართვა;
- `QgsFeature` — გეოგრაფიული ობიექტის წარმოდგენა;
- `QgsGeometry` — გეომეტრიული მონაცემების წარმოდგენა და დამუშავება.

ეს კლასები QGIS-ის C++ API-ს შეესაბამება და Python-იდან მათი გამოყენება SIP-ის მიერ გენერირებული დამაკავშირებელი კოდის მეშვეობით ხდება.

მნიშვნელოვანია: SIP თავად არ არის PyQGIS და არც QGIS-ის ბირთვი. ის წარმოადგენს ერთ-ერთ ტექნოლოგიას, რომლის საშუალებითაც C++-ზე დაფუძნებული QGIS-ის ფუნქციონალი Python-იდან ხელმისაწვდომი ხდება.

> **გზამკვლევი ორ ენაზე**
> წარმოიდგინე საქართველოს რუკა ორ ენაზე. ქალაქები და გზები ერთი და იგივეა, განსხვავდება მხოლოდ წარწერების ენა. PyQGIS არის „ქართული წარწერებით“ რუკა, C++ API — „ორიგინალის“.

საბოლოოდ, თუ PyQGIS-ში იცი რა უნდა გააკეთო, C++-ში ახალი ლოგიკა კი არ გჭირდება, არამედ ახალი **გრამატიკა**: ტიპები, მაჩვენებლები, კომპილაცია.

01.2 QGIS-ის არქიტექტურა

QGIS-ის წყარო კოდი რამდენიმე ბიბლიოთეკადაა დაყოფილი:

საქაღალდე	ბიბლიოთეკა	რას შეიცავს	გეო-მაგალითი
`src/core/`	`qgis_core`	გეომეტრია, CRS, ფენები, ატრიბუტები, რენდერი	`QgsGeometry`, `QgsCoordinateReferenceSystem`
`src/gui/`	`qgis_gui`	ვიჯეტები და დიალოგები	ფენის არჩევის კომბობოქსი
`src/analysis/`	`qgis_analysis`	გეოპროცესინგი და ანალიზი	ბუფერი, გადაკვეთა
`src/app/`	`qgis`	თავად დესკტოპ პროგრამა	მენიუ, ინსტრუმენტთა პანელი

> **დაიმახსოვრე**
> `core` არ იცნობს ინტერფეისს, `gui` კი `core`-ს ეყრდნობა. ამიტომაც შეგიძლია `core` ფანჯრების გარეშე, სერვერზეც გამოიყენო.

01.3 ერთი და იგივე ამოცანა ორ ენაზე

ამოცანა: ფენის ყველა ობიექტის სახელის გამოტანა.

PYTHON

```python
layer = iface.activeLayer()
for feature in layer.getFeatures():
    print(feature["name"])
```

C++

```cpp
QgsVectorLayer *layer =
  qobject_cast<QgsVectorLayer *>
(iface->activeLayer());
QgsFeatureIterator it = layer-
>getFeatures();
QgsFeature feature;
while (it.nextFeature(feature)) {
  qDebug() <<
feature.attribute("name").toString(
);
}
```

რა არის იგივე: `getFeatures`, `attribute`. რა არის ახალი: ტიპების სახელები (`QgsVectorLayer`), `*` ნიშანი, `;` ყოველი ხაზის ბოლოს, `qobject_cast`. თითოეულს ამ კურსში ცალკე გავარჩევთ.

01.4 პირველი C++ პროგრამა

C++ პროგრამა კომპილდება: წყარო კოდს კომპილატორი გადააქცევს მანქანურ ფაილად, რომელსაც შემდეგ ვუშვებთ. Python-ში ეს ეტაპი არ გვქონდა.

C++ main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, Georgia!" << std::endl;
    return 0;
}
```

SHELL

```
g++ main.cpp -o hello
./hello
```

შედეგი

```
Hello, Georgia!
```

ნაწილები

`#include` ჩართავს ბიბლიოთეკას, `main()` არის პროგრამის შესასვლელი წერტილი, `std::cout` გამოიტანს ტექსტს ეკრანზე.

01.5 Python vs C++ ერთი შეხედვით

	Python / PyQGIS	C++ / QGIS Core
გაშვება	ინტერპრეტატორი, პირდაპირ	კომპილაცია, შემდეგ გაშვება
ტიპები	დინამიური: `x = 5`	სტატიკური: `int x = 5;`
მეხსიერება	ავტომატური	ხელით, ან smart pointer-ებით
შეცდომები	გაშვების დროს	უმეტესად კომპილაციისას
სიჩქარე	მოხერხებული	მაღალი
კოდის დაცვა	`.py` ღია ტექსტია	შედეგი ბინარულია

01.6 სავარჯიშო და შეჯამება

სავარჯიშო

1. გახსენი `api.qgis.org` და მოძებნე კლასი `QgsFeature`. ჩამოწერე სამი მეთოდი, რომლებიც PyQGIS-იდან უკვე იცი.
2. დაწერე და გაუშვი „Hello“ პროგრამა, ოღონდ გამოიტანე შენი ქალაქის სახელი.
3. დაამატე მეორე ხაზი: `Population: 1100000` (თბილისის მიახლოებითი მოსახლეობა).

შეჯამება

- PyQGIS და C++ API ერთი და იგივე კლასებია სხვადასხვა „გრამატიკით“.
- `core` — ლოგიკა, `gui` — ინტერფეისი, `app` — პროგრამა.
- C++ კოდი იკომპილება და მკაცრად ტიპიზირებულია.

CHAPTER 02

02

ტიპები და ცვლადები

Types and variables

int · double · bool · auto · const · ტიპების გარდაქმნა · კოორდინატები და სიზუსტე

Python-ში `x = 5` საკმარისია. C++-ში კომპილატორს წინასწარ უნდა ეცოდინებოდეს, რა სახის მონაცემია ცვლადში. ამიტომ QGIS-ის დოკუმენტაციაში ყველგან ტიპები წერია.

02.1 ცვლადი როგორც ეტიკეტიანი ყუთი

ყუთი და ეტიკეტი

Python-ში ყუთს არ აქვს ფორმა: ერთსა და იმავე ყუთში ჯერ რიცხვს ჩადებ, მერე ტექსტს. C++-ში ყუთი კონკრეტული ფორმისაა: `int`-ის ყუთში მხოლოდ მთელი რიცხვი ჯდება, `double`-ის ყუთში ათწილადი. ეს იმიტომაა მკაცრი, რომ კომპიუტერმა წინასწარ იცოდეს, რამდენ მეხსიერებას დაიკავებს.

PYTHON

```python
lat = 41.7151
vertex_count = 12
is_valid = True
```

C++

```cpp
double lat = 41.7151;
int vertexCount = 12;
bool isValid = true;
```

02.2 ძირითადი ტიპები

ტიპი	რას ინახავს	გეო-მაგალითი
`int`	მთელი რიცხვი	`int vertexCount = 12;`
`double`	ათწილადი (ზუსტი)	`double lat = 41.7151;`
`bool`	`true` / `false`	`bool isValid = true;`
`char`	ერთი სიმბოლო	`char hemisphere = 'N';`
`std::string`	ტექსტი	`std::string city = "Batumi";`
`qint64` (Qt)	დიდი მთელი რიცხვი	ფენაში მილიონობით ობიექტი

კოორდინატებისთვის

კოორდინატებს ყოველთვის `double`-ში ვინახავთ. QGIS-ის `QgsPointXY` შიგნით სწორედ ორ `double`-ს შეიცავს.

02.3 კოდი შედეგით

C++ coords.cpp

```cpp
#include <iostream>

int main() {
    double lat = 41.7151;
    double lon = 44.8271;
    int vertexCount = 12;
    bool isValid = true;

    std::cout << "Tbilisi: " << lat << ", " << lon << "\n";
    std::cout << "Vertices: " << vertexCount << "\n";
    std::cout << std::boolalpha << "Valid: " << isValid << "\n";
    return 0;
}
```

შედეგი

```
Tbilisi: 41.7151, 44.8271
Vertices: 12
Valid: true
```

`std::boolalpha` გვაიძულებს, `bool` გამოვიტანოთ სიტყვებით. მის გარეშე დაიბეჭდებოდა `1`.

02.4 const და auto

`const` ნიშნავს, რომ მნიშვნელობას შემდეგ აღარ შევცვლით. `auto` კომპილატორს უთხრებს, რომ ტიპი თვითონ გამოიცნოს მარჯვენა მხარიდან.

C++

```cpp
const double EARTH_RADIUS_KM = 6371.0;   // არ იცვლება
auto area = 50.0;                         // double
auto count = 12;                          // int
```

> **ყურადღება**
> `const` ცვლადს მნიშვნელობა მხოლოდ შექმნისას ეძლევა. მოგვიანებით ცვლილების მცდელობა კომპილაციის შეცდომას იძლევა და ეს კარგია: შეცდომა კოდის გაშვებამდე ჩანს.

02.5 მთელი რიცხვების გაყოფა

ეს პირველი ხაფანგია Python-იდან მოსულებისთვის.

C++ division.cpp

```cpp
#include <iostream>

int main() {
    int totalHectares = 7;
    int parcels = 2;

    std::cout << totalHectares / parcels << "\n";
    std::cout << totalHectares / 2.0 << "\n";
    return 0;
}
```

შედეგი

```
3
3.5
```

ორი `int`-ის გაყოფა `int`-ს იძლევა და წილადი ნაწილი იკარგება (Python-ში `7 / 2` იყო `3.5`, ხოლო `7 // 2` — `3`). თუ ერთი მხარე `double`-ა, შედეგიც `double` იქნება.

02.6 სიზუსტე და გარდაქმნა

ათწილადებს ზუსტად ვერ ვადარებთ ერთმანეთს `==`-ით, რადგან კომპიუტერი ორობით სისტემაში ინახავს.

C++ precision.cpp

```cpp
#include <iostream>
#include <cmath>

int main() {
    double a = 0.1 + 0.2;
    std::cout << std::boolalpha;
    std::cout << (a == 0.3) << "\n";
    std::cout << (std::abs(a - 0.3) < 1e-9) << "\n";

    int whole = static_cast<int>(3.99);
    std::cout << whole << "\n";
    return 0;
}
```

შედეგი

```
false
true
3
```

წესი

ათწილადები ვადარო სიზუსტით (`abs(a - b) < epsilon`). `static_cast<int>(...)` აკეთებს აშკარა გარდაქმნას და ათწილადი ნაწილი უბრალოდ იკვეთება.

02.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. გამოაცხადე ცვლადები ბათუმისთვის: განედი `41.6168`, გრძედი `41.6367`, მოსახლეობა `int`-ად (მიახლოებით 170000). გამოიტანე ისინი.
2. წარმოიდგინე 5 ნაკვეთი, საერთო ფართობი 12 ჰექტარი. გამოთვალე საშუალო ფართობი სწორად (`double`).
3. რას დაბეჭდავს `static_cast<int>(-2.7)` ? შეამოწმე.

შეჯამება

- ტიპი ცვლადის „ყუთის ფორმაა“: `int`, `double`, `bool`.
- კოორდინატები — `double`, რაოდენობები — `int` ან `qint64`.
- `int / int` იძლევა `int`-ს.
- ათწილადებს ვადარებთ ზღვრით, არა `==`-ით.

CHAPTER 03

03

ფუნქციები და namespace

Functions and namespaces

ფუნქციის ხელმოწერა · overload · default არგუმენტი · .h/.cpp დაყოფა · namespace და ::

QGIS-ის კოდში ყველაფერი კლასებშია, მაგრამ კლასის მეთოდიც ფუნქციაა. ფუნქციის ხელმოწერის კითხვა დოკუმენტაციის გასაგებად პირველი აუცილებელი უნარია.

03.1 ფუნქციის ანატომია

```C++
double calculateArea(double width, double height) {
    return width * height;
}
```

ნაწილი	მაგალითში	მნიშვნელობა
დაბრუნების ტიპი	`double`	რას დაგიბრუნებს
სახელი	`calculateArea`	როგორ გამოვიძახოთ
პარამეტრები	`double width, double height`	რა უნდა გადავცეთ (თითოეულს ტიპი აქვს)
სხეული	`{ ... }`	რას აკეთებს

თუ ფუნქცია არაფერს აბრუნებს, დაბრუნების ტიპია `void`.

> **გავრცელებული შეცდომა**
> ფუნქციის განსაზღვრისას ფრჩხილების შემდეგ `;` არ დაიდება: `double f(double a); { ... }` არასწორია, რადგან `;` აქ ფუნქციას „დეკლარაციად“ აქცევს და ფიგურული ფრჩხილები ფუნქციას აღარ ეკუთვნის.

03.2 namespace — სახელების საქაღალდე

> **საქაღალდეები კომპიუტერში**
> ორ საქაღალდეში (`ProjectA`, `ProjectB`) შეიძლება ერთი სახელის ფაილი გვქონდეს, `data.txt`. კონფლიქტი არ იქნება, რადგან მისამართი განსხვავებულია. `namespace` სწორედ ასეთი საქაღალდეა კოდისთვის.

C++ area.cpp

```cpp
#include <iostream>

namespace GIS {
    double calculateArea(double width, double height) {
        return width * height;
    }
}

int main() {
    double area = GIS::calculateArea(5.0, 10.0);
    std::cout << "Area: " << area << std::endl;
    return 0;
}
```

შედეგი

```
Area: 50
```

`::` არის scope resolution operator: „შედი `GIS`-ში და იქიდან აიღე `calculateArea`“. ანალოგიურად `std::cout` ნიშნავს „სტანდარტული ბიბლიოთეკის `cout`“.

> **QGIS-ში**
>
> `Qgis::WkbType`, `Qgis::DistanceUnit` — ეს `Qgis` სახელების სივრცეა. კლასებს კი `Qgs` პრეფიქსი აქვთ: `QgsFeature`, `QgsProject`. პრეფიქსიც ერთგვარი namespace-ია ძველი სტილით.

03.3 overload — ერთი სახელი, სხვადასხვა პარამეტრები

C++-ში ერთსა და იმავე სახელზე რამდენიმე ფუნქცია შეიძლება არსებობდეს, თუ პარამეტრები განსხვავდება.

C++ overload.cpp

```cpp
#include <iostream>

namespace GIS {
    double calculateArea(double width, double height) {
        return width * height;                // მართკუთხედი
    }
    double calculateArea(double radius) {
        return 3.14159 * radius * radius;     // წრე
    }
}

int main() {
    std::cout << GIS::calculateArea(5.0, 10.0) << "\n";
    std::cout << GIS::calculateArea(10.0) << "\n";
    return 0;
}
```

შედეგი

```
50
314.159
```

QGIS-ის API სავსეა ასეთი overload-ებით: მაგალითად, `QgsPointXY` რამდენიმე კონსტრუქტორით იქმნება.

03.4 default პარამეტრი

C++ units.cpp

```cpp
#include <iostream>

double toMeters(double value, double factor = 1.0) {
    return value * factor;
}

int main() {
    std::cout << toMeters(2.5, 1000.0) << "\n";   // კილომეტრი -> მეტრი
    std::cout << toMeters(7.0) << "\n";            // უკვე მეტრებშია
    return 0;
}
```

შედეგი

```
2500
7
```

default მნიშვნელობა მხოლოდ ბოლო პარამეტრებს შეიძლება ჰქონდეთ.

03.5 მანძილი ორ წერტილს შორის

C++ distance.cpp

```cpp
#include <iostream>
#include <cmath>

namespace GIS {
    double distance(double x1, double y1, double x2, double y2) {
        return std::hypot(x2 - x1, y2 - y1);
    }
}

int main() {
    std::cout << GIS::distance(0.0, 0.0, 3.0, 4.0) << "\n";
    return 0;
}
```

შედეგი

```
5
```

`std::hypot(a, b)` არის `sqrt(a*a + b*b)`, ანუ პითაგორას თეორემა: მანძილი დეკარტულ (გეგმურ) კოორდინატებში.

03.6 .h და .cpp ფაილები

დიდ პროექტში ფუნქციის **დეკლარაცია** (რას აკეთებს) და **განსაზღვრება** (როგორ აკეთებს) ცალკე ფაილებშია. QGIS-ის მთელი წყარო კოდი ასეა აგებული, `api.qgis.org` კი სწორედ `.h` ფაილებიდან გენერირდება.

C++ gis_utils.h

```cpp
#pragma once

namespace GIS {
    double distance(double x1, double y1, double x2, double y2);
}
```

C++ gis_utils.cpp

```cpp
#include "gis_utils.h"
#include <cmath>

double GIS::distance(double x1, double y1, double x2, double y2) {
    return std::hypot(x2 - x1, y2 - y1);
}
```

C++ main.cpp

```cpp
#include <iostream>
#include "gis_utils.h"

int main() {
    std::cout << GIS::distance(0.0, 0.0, 3.0, 4.0) << "\n";
}
```

SHELL

```
g++ main.cpp gis_utils.cpp -o gis
```

შენიშვნა

`#include <...>` — სისტემური ბიბლიოთეკა, `#include "..."` — შენი საკუთარი ფაილი. `#pragma once` იცავს ფაილს ორჯერ ჩართვისგან.

03.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაწერე `GIS::perimeter(double width, double height)` (მართკუთხა ნაკვეთის პერიმეტრი) და შეამოწმე 5 × 10 ზომაზე.
2. დაამატე overload კვადრატისთვის: `perimeter(double side)`.
3. გამოთვალე მანძილი წერტილებს შორის (2, 3) და (6, 6).
4. გადაიტანე ფუნქციები `gis_utils.h` / `gis_utils.cpp` ფაილებში.

შეჯამება

- ფუნქცია = დაბრუნების ტიპი + სახელი + ტიპიზებული პარამეტრები.
- `namespace` იცავს სახელებს კონფლიქტისგან, `::` მასში შედის.
- overload და default პარამეტრები QGIS API-ში ყველგანაა.
- `.h` — დეკლარაცია, `.cpp` — განსაზღვრება.

CHAPTER 04

04

მაჩვენებლები (Pointers)

Pointers

მისამართი და მნიშვნელობა · & და * · -> · nullptr · ჩამოკიდებული მაჩვენებელი

ეს ყველაზე მნიშვნელოვანი თემაა. QGIS-ის API-ში ყოველ მეორე ფუნქციას აქვს `*` ან `->`: `QgsMapLayer *layer`, `layer->name()`.

04.1 კურიერი და მისამართი

> **კურიერი**
> კურიერს მძიმე ამანათი მოუვიდა. ის მას ადრესატთან კი არ მიაქვს, არამედ ეუბნება: „ამანათი ამ საწყობის ამ მისამართზეა“. მაჩვენებელი არის სწორედ **მისამართი**, არა თავად ამანათი. ბევრად იაფია მისამართის გადაცემა, ვიდრე მილიონობით წერტილიანი ფენის კოპირება.

ორი სიმბოლო:

სიმბოლო	სახელი	რას აკეთებს
`&x`	address-of	აბრუნებს `x`-ის მისამართს
`*p`	dereference	გადადის მისამართზე და კითხულობს მნიშვნელობას

04.2 პირველი მაჩვენებელი

C++ ptr.cpp

```cpp
#include <iostream>

int main() {
    int layerCount = 5;
    int *ptr = &layerCount;

    std::cout << "Layer count: " << layerCount << "\n";
    std::cout << "Address: " << ptr << "\n";
    std::cout << "Value at address: " << *ptr << "\n";
    return 0;
}
```

შედეგი

```
Layer count: 5
Address: 0x7ffd5c3a1a24
Value at address: 5
```

> **შენიშვნა**
> მისამართი ყოველ გაშვებაზე და ყოველ კომპიუტერზე განსხვავებულია (Windows-ზე ის ასე გამოიყურება: `000000BF7C3BF970`). მნიშვნელობა კი ერთი და იგივეა.

სამი ხაზი = სამი ფაქტი: **რა ძევს** (`5`), **სად ძევს** (მისამართი), **რას ვპოულობთ იქ** (`*ptr`).

04.3 მაჩვენებელი ობიექტზე: ისარი

მაჩვენებელი ხშირად მიუთითებს მთელ ობიექტზე. წევრთან მისასვლელად წერტილის ნაცვლად `->` გამოიყენება.

C++ layer_ptr.cpp

```cpp
#include <iostream>
#include <string>

struct Layer {
    std::string name;
    int featureCount;
};

int main() {
    Layer rivers{"Rivers", 42};
    Layer *ptr = &rivers;

    std::cout << ptr->name << ": " << ptr->featureCount << "\n";

    ptr->featureCount += 8;                       // იგივეა, რაც
(*ptr).featureCount += 8
    std::cout << rivers.featureCount << "\n";
    return 0;
}
```

შედეგი

```
Rivers: 42
50
```

ხედავ? `ptr`-ით ცვლილება პირდაპირ `rivers`-ს შეეხო, რადგან ორივე ერთსა და იმავე ობიექტზე მიუთითებს, კოპირება არ მომხდარა.

> **წესი**
> ობიექტზე — `.` (`rivers.name`), მაჩვენებლით — `->` (`ptr->name`). ზუსტად ამიტომ გვაქვს QGIS-ში `layer->featureCount()`.

04.4 nullptr — „ფენა არ არის არჩეული“

მაჩვენებელი შეიძლება არაფერზე მიუთითებდეს. ამას `nullptr` ჰქვია.

C++ nullptr.cpp

```cpp
#include <iostream>
#include <string>

struct Layer {
    std::string name;
    int featureCount;
};

int main() {
    Layer *activeLayer = nullptr;

    if (activeLayer == nullptr) {
        std::cout << "No active layer\n";
    } else {
        std::cout << activeLayer->name << "\n";
    }
    return 0;
}
```

შედეგი

```
No active layer
```

ეს QGIS-ის ყოველდღიური სიტუაციაა: `iface->activeLayer()` აბრუნებს `nullptr`-ს, თუ ფენა არ არის მონიშნული. PyQGIS-ში ეს `None` იყო.

PYTHON

```python
layer = iface.activeLayer()
if layer is None:
    print("No active layer")
```

C++

```cpp
QgsMapLayer *layer = iface->activeLayer();
if (layer == nullptr) {
  qDebug() << "No active layer";
}
```

ხშირი შეცდომა

`nullptr`-ზე `->` გამოძახება პროგრამას ამსხვრევს (segmentation fault / crash). მაჩვენებელი ყოველთვის უნდა შეამოწმო `nullptr`-ზე, სანამ მას გამოიყენებ.

04.5 ჩამოკიდებული მაჩვენებელი

საშიში ადგილი

თუ ობიექტი აღარ არსებობს, მაჩვენებელი მაინც ინახავს ძველ მისამართს. ეს ჰგავს კურიერს, რომელიც მისამართზე მიდის, სადაც სახლი უკვე დაანგრიეს.

C++ `dangling.cpp`

```cpp
int *makeCounter() {
    int count = 10;
    return &count;      // შეცდომა: count ფუნქციის დასრულებისას იშლება
}
```

`count` ლოკალური ცვლადია და ფუნქციის დასრულებისას იშლება. დაბრუნებული მისამართი უკვე არავის ეკუთვნის. ამის თავიდან ასაცილებლად არსებობს smart pointer-ები (თავი 08).

04.6 სავარჯიშო და შეჯამება

სავარჯიშო

1. შექმენი `struct Parcel { std::string owner; double area; };`. გააკეთე მაჩვენებელი და `->`-ით გამოიტანე მფლობელი და ფართობი.
2. ფართობი გაზარდე 100 კვ.მ-ით მაჩვენებლის საშუალებით და შეამოწმე თავდაპირველ ობიექტში.
3. შექმენი `Parcel *p = nullptr;` და დაამატე შემოწმება, რომ პროგრამა არ დაიმსხვრეს.

შეჯამება

- მაჩვენებელი ინახავს **მისამართს**; `&` მისამართს იღებს, `*` მნიშვნელობას კითხულობს.
- ობიექტზე მაჩვენებლით ვიყენებთ `->`-ს.
- `nullptr` = „არაფერზე მიუთითებს“; ყოველთვის შეამოწმე გამოყენებამდე.
- ლოკალური ცვლადის მისამართს ფუნქციიდან არ აბრუნებენ.

CHAPTER 05

05

მითითებები (References) და const

References and const

& როგორც მეტსახელი · გადაცემა მნიშვნელობით და მითითებით · const & · QGIS ხელმოწერების კითხვა

წინა თავში მაჩვენებელი გავიგეთ. QGIS-ის API-ში მასთან ერთად ყველგან გვხვდება `&`: `const QgsFeature &feature`. ამ თავის შემდეგ ამ ჩანაწერს დაუფიქრებლად წაიკითხავ.

05.1 მეტსახელი

მითითება (reference) არის ცვლადის **მეტსახელი**: ერთი ობიექტი, ორი სახელი.

C++ alias.cpp

```cpp
#include <iostream>

int main() {
    int featureCount = 100;
    int &ref = featureCount;     // ref არის featureCount-ის მეტსახელი

    ref = 300;
    std::cout << featureCount << "\n";
    return 0;
}
```

შედეგი

```
300
```

> **ბანკის პერსონალური ასისტენტი**
> სოლო მომსახურების ასისტენტი ერთხელ და სამუდამოდ გიმაგრდება. ის არ იცვლება ყოველ ვიზიტზე და ცარიელი არ არის. მითითება ზუსტად ასეა: იბადება კონკრეტულ ობიექტზე მიბმული, ვეღარ გადაიბმება სხვაზე და `nullptr` ვერ იქნება.

05.2 მაჩვენებელი თუ მითითება?

	მაჩვენებელი `*`	მითითება `&`
შეიძლება ცარიელი იყოს	დიახ, `nullptr`	არა
შეიძლება სხვაზე გადაიბას	დიახ	არა
წევრთან მისვლა	`p->name`	`r.name`
როდის	ობიექტი შეიძლება არ არსებობდეს	ობიექტი აუცილებლად არსებობს

მოკლედ

მაჩვენებელი — როცა ობიექტი შეიძლება არ არსებობდეს ან შეიცვალოს.
მითითება — როცა ობიექტი უეჭველად არსებობს.

05.3 ფუნქციაში გადაცემა: კოპირება თუ

ორიგინალი

C++ byref.cpp

```cpp
#include <iostream>
#include <string>

struct Layer {
    std::string name;
    int featureCount;
};

void addByValue(Layer layer, int n) {
    layer.featureCount += n;          // ცვლის ასლს
}

void addByRef(Layer &layer, int n) {
    layer.featureCount += n;          // ცვლის ორიგინალს
}

int main() {
    Layer roads{"Roads", 100};

    addByValue(roads, 25);
    std::cout << "After value: " << roads.featureCount << "\n";

    addByRef(roads, 25);
    std::cout << "After ref: " << roads.featureCount << "\n";
    return 0;
}
```

შედეგი

```
After value: 100
After ref: 125
```

პირველ შემთხვევაში ფუნქციამ ობიექტის **ასლი** მიიღო და ორიგინალი არ შეცვლილა. მეორეში `&`-ის გამო იმუშავა თავად ორიგინალზე.

05.4 const & — უსაფრთხო დათვალიერება

თუ ფუნქციას მხოლოდ წასაკითხად სჭირდება ობიექტი, ვწერთ `const T &`. კოპირება არ ხდება და ფუნქციას ცვლილების უფლებაც არ აქვს.

C++ perimeter.cpp

```cpp
#include <iostream>
#include <vector>
#include <cmath>

struct Point {
    double x;
    double y;
};

double perimeter(const std::vector<Point> &ring) {
    double total = 0.0;
    for (size_t i = 0; i < ring.size(); ++i) {
        const Point &a = ring[i];
        const Point &b = ring[(i + 1) % ring.size()];
        total += std::hypot(b.x - a.x, b.y - a.y);
    }
    return total;
}

int main() {
    std::vector<Point> parcel = {{0, 0}, {10, 0}, {10, 10}, {0, 10}};
    std::cout << "Perimeter: " << perimeter(parcel) << "\n";
    return 0;
}
```

შედეგი

```
Perimeter: 40
```

შენიშვნა

`(i + 1) % ring.size()` ბოლო წერტილიდან პირველს უკავშირებს ანუ კონტურს ხურავს. თუ გადაიცემოდა `std::vector<Point> ring` (&-ის გარეშე), ყოველ გამოძახებაზე მთელი წერტილების სია დაკოპირდებოდა.

ყურადღება

`const`-ის დარღვევა კომპილაციის შეცდომაა. `ring[0].x = 5;` ამ ფუნქციაში კომპილაციას ვერ გაივლის.

05.5 QGIS-ის ხელმოწერების კითხვა

ახლა `api.qgis.org`-ის გვერდებიდან ეს ჩანაწერები ადვილად იკითხება:

ჩანაწერი	როგორ წავიკითხო
`QgsVectorLayer *layer`	ფენის მისამართი; შეიძლება `nullptr` იყოს, შეამოწმე
`const QgsFeature &feature`	ობიექტი აუცილებლად არსებობს, ვკითხულობ და არ ვცვლი, არ ვაკოპირებ
`QgsGeometry &geometry`	ობიექტი არსებობს და ფუნქციას უფლება აქვს, ის შეცვალოს
`const QString &name`	ტექსტი კოპირების გარეშე, მხოლოდ საკითხავად
`QgsFeature feature`	მიიღება ასლი

PYTHON

```python
def count_features(layer):
    return layer.featureCount()
```

C++

```cpp
qint64 countFeatures(const
QgsVectorLayer *layer) {
  return layer->featureCount();
}
```

05.6 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაწერე `void expand(Parcel &p, double extra)`, რომელიც ნაკვეთის ფართობს გაზრდის. შეამოწმე `main`-ში.
2. დაწერე `void printParcel(const Parcel &p)`. შეეცადე მასში ფართობის შეცვლას და ნახე კომპილატორის შეცდომა.
3. დაწერე `double area(const std::vector<Point> &ring)` (შუსტერის ფორმულა: `0.5 * |Σ (xᵢ·yᵢ₊₁ − xᵢ₊₁·yᵢ)|`). კვადრატული ნაკვეთისთვის (0,0), (10,0), (10,10), (0,10) უნდა მიიღო 100.

შეჯამება

- `T &` — მეტსახელი, ობიექტი აუცილებლად არსებობს.
- ფუნქციაში ობიექტს ორიგინალად გადავცემთ `&`-ით, წასაკითხად — `const &`-ით.
- `T *` — მაჩვენებელი, `nullptr`-ს შემოწმებით.
- ახლა `const QgsFeature &feature` სრულად გესმის.

CHAPTER 06

06

კლასები და ობიექტები

Classes and objects

class და struct · public/private · კონსტრუქტორი · const მეთოდები · QGIS-ის სახელდების წესი

QGIS-ში თითქმის ყველაფერი კლასია: `QgsPointXY`, `QgsFeature`, `QgsVectorLayer`. კლასი აერთიანებს **მონაცემს** (წევრ-ცვლადებს) და **ქცევას** (მეთოდებს).

06.1 კლასი და ობიექტი

ნახაზი და სახლი

კლასი არის ნახაზი, ობიექტი კი ამ ნახაზით აშენებული სახლი. ერთი ნახაზით შეიძლება ბევრი სახლი აშენდეს: ერთი კლასი `Parcel`, ბევრი კონკრეტული ნაკვეთი.

PYTHON

```python
class Parcel:
    def __init__(self, owner, w,
h):
        self.owner = owner
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h
```

C++

```cpp
class Parcel {
public:
  Parcel(std::string owner, double
w, double h);
  double area() const;
private:
  std::string mOwner;
  double mW, mH;
};
```

06.2 public და private

`private` წევრებს მხოლოდ თავად კლასის მეთოდები ხედავენ. გარედან მათზე წვდომა კომპილაციის შეცდომაა. ასე ობიექტი საკუთარ მდგომარეობას იცავს და გარე კოდი მას არასწორ მდგომარეობაში ვერ ჩააგდებს.

> **QGIS-ის წესი**
> წევრ-ცვლადებს აქვთ `m` პრეფიქსი (`mOwner`, `mWidth`), მეთოდები კი `get`-ის გარეშეა: `feature.id()`, არა `feature.getId()`. შეცვლისთვის გამოიყენება `set`: `feature.setGeometry(...)`.

06.3 სრული მაგალითი

C++ parcel.cpp

```cpp
#include <iostream>
#include <string>
#include <utility>

class Parcel {
public:
    Parcel(std::string owner, double width, double height)
        : mOwner(std::move(owner)), mWidth(width), mHeight(height) {}

    double area() const { return mWidth * mHeight; }
    const std::string &owner() const { return mOwner; }

    void resize(double width, double height) {
        mWidth = width;
        mHeight = height;
    }

private:
    std::string mOwner;
    double mWidth;
    double mHeight;
};

int main() {
    Parcel p("Nino", 20.0, 15.0);
    std::cout << p.owner() << ": " << p.area() << " m2\n";

    p.resize(25.0, 20.0);
    std::cout << p.owner() << ": " << p.area() << " m2\n";
    return 0;
}
```

შედეგი

```
Nino: 300 m2
Nino: 500 m2
```

06.4 კონსტრუქტორი

კონსტრუქტორი არის სპეციალური მეთოდი, რომელიც ობიექტის შექმნისას სრულდება. მას სახელი კლასის სახელი აქვს და დაბრუნების ტიპი არა. წევრებს ვიწყებთ ე.წ. **ინიციალიზაციის სიით** (`: mOwner(...), mWidth(width)`), ეს მიღებული წესია.

C++

```cpp
Parcel p("Nino", 20.0, 15.0);          // ობიექტი შეიქმნა
```

QGIS-ის მაგალითი:

C++

```cpp
QgsPointXY tbilisi(44.8271, 41.7151);   // x = გრძედი(lon), y = განედი(lat)
double x = tbilisi.x();
```

> **გაითვალისწინე**
> QGIS-ში წერტილი `(x, y)` = `(გრძედი, განედი)`, ანუ `(lon, lat)`. ბევრ რუკის API-ში პირიქითაა. შეცდომა ძალიან გავრცელებულია.

06.5 const მეთოდი

`double area() const` — ბოლოს `const` ნიშნავს, რომ მეთოდი ობიექტს არ ცვლის. ამიტომ ის `const Parcel &`-ზეც გამოიძახება. გახსოვდეს `const QgsFeature &feature`: მასზე მხოლოდ `const` მეთოდები გამოიძახება.

მეთოდი	`const`?	რატომ
`area()`	დიახ	მხოლოდ კითხულობს
`owner()`	დიახ	მხოლოდ კითხულობს
`resize(...)`	არა	ცვლის ობიექტს

06.6 struct და class

C++-ში `struct` და `class` თითქმის ერთია. ერთადერთი განსხვავება: `struct`-ში წევრები ნაგულისხმევად `public`-ია, `class`-ში `private`. პრაქტიკაში `struct` მარტივ მონაცემებს (`Point`) ვიყენებთ, `class` — ქცევიან ობიექტებს.

06.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაამატე `Parcel`-ს მეთოდი `double perimeter() const`.
2. დაამატე `bool isLargerThan(const Parcel &other) const`.
3. შექმენი კლასი `Region`, რომელსაც აქვს სახელი და მოსახლეობა, და მეთოდი `density(double areaKm2) const`.
4. შეეცადე `p.mWidth = 5;`-ს `main`-ში და წაიკითხე კომპილატორის შეტყობინება.

შეჯამება

- კლასი = მონაცემი + მეთოდები; ობიექტი = კლასის კონკრეტული ეგზემპლარი.
- `private` იცავს მდგომარეობას, `public` არის ინტერფეისი.
- QGIS: `mMember`, `name()`, `setName()`.
- `const` მეთოდი ობიექტს არ ცვლის.

CHAPTER 07

07

მემკვიდრეობა და პოლიმორფიზმი

Inheritance and polymorphism

ბაზისური და მემკვიდრე კლასი · virtual / override · dynamic_cast · qobject_cast

ფენების იერარქია QGIS-ის ხერხემალია: `QgsMapLayer` ზოგადი ფენაა, `QgsVectorLayer` და `QgsRasterLayer` მისი კონკრეტული სახეები. ეს თავი ხსნის, როგორ მუშაობს ასეთი სტრუქტურა.

07.1 მემკვიდრეობა

ზოგადი და კონკრეტული

„ტრანსპორტი“ ზოგადი ცნებაა, „ავტობუსი“ და „მატარებელი“ მისი სახეებია. ორივეს აქვს სიჩქარე და მარშრუტი, მაგრამ თითოეულს თავისი თვისებაც აქვს. ფენებიც ასეა: ყველას აქვს სახელი და CRS, ვექტორულს დამატებით ობიექტები, რასტრულს — არხები (band).

```
QgsMapLayer
 ├─ QgsVectorLayer   (ობიექტები, ატრიბუტები)
 └─ QgsRasterLayer   (პიქსელები, არხები)
```

C++-ში მემკვიდრეობა იწერება `: public Base`.

07.2 virtual და override

თუ მეთოდი `virtual`-ია, გამოძახებისას ირჩევა **რეალური** ობიექტის ვერსია და არა მაჩვენებლის ტიპისა. ამას პოლიმორფიზმი ჰქვია.

C++ layers.cpp

```cpp
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class Layer {
public:
    explicit Layer(std::string name) : mName(std::move(name)) {}
    virtual ~Layer() = default;

    const std::string &name() const { return mName; }
    virtual std::string type() const { return "generic"; }

private:
    std::string mName;
};

class VectorLayer : public Layer {
public:
    VectorLayer(std::string name, int count)
        : Layer(std::move(name)), mCount(count) {}

    std::string type() const override { return "vector"; }
    int featureCount() const { return mCount; }

private:
    int mCount;
};

class RasterLayer : public Layer {
public:
    explicit RasterLayer(std::string name) : Layer(std::move(name)) {}
    std::string type() const override { return "raster"; }
```

```
};

int main() {
    VectorLayer rivers("Rivers", 42);
    RasterLayer dem("DEM");

    std::vector<Layer *> layers = {&rivers, &dem};
    for (Layer *layer : layers) {
        std::cout << layer->name() << " is " << layer->type() << "\n";
    }
    return 0;
}
```

შედეგი

```
Rivers is vector
DEM is raster
```

ორივე ელემენტი `Layer *`-ია, მაგრამ `type()` თითოეულისთვის საკუთარ პასუხს აბრუნებს. სწორედ ამიტომ ინახავს QGIS ყველა ფენას ერთ სიაში, როგორც `QgsMapLayer *`.

> **შენიშვნა**
>
> `override` არ არის სავალდებულო, მაგრამ კომპილატორს ამოწმებს: თუ მეთოდის სახელი ან პარამეტრი შეეშალა, შეცდომას დაგიბრუნებს. `virtual ~Layer()` ბაზისურ კლასში აუცილებელია, რომ წაშლისას სწორი დესტრუქტორი გამოიძახოს.

07.3 ტიპის გარდაქმნა: dynamic_cast

სიაში `Layer *` გვაქვს. `featureCount()` კი მხოლოდ `VectorLayer`-ს აქვს. როგორ მივწვდეთ? `dynamic_cast` ამოწმებს რეალურ ტიპს გაშვების დროს.

C++

```
for (Layer *layer : layers) {
    if (auto *vec = dynamic_cast<VectorLayer *>(layer)) {
        std::cout << vec->name() << ": " << vec->featureCount() << "
features\n";
    }
}
```

შედეგი

```
Rivers: 42 features
```

თუ ობიექტი ვექტორული ფენა არ არის, `dynamic_cast` აბრუნებს `nullptr`-ს. ზუსტად ამიტომ ვამოწმებთ `if`-ში.

07.4 qobject_cast QGIS-ში

QGIS-ის კლასები `QObject`-იდან მოდის, ამიტომ იქ `dynamic_cast`-ის უფრო სწრაფი ვერსია გამოიყენება:

PYTHON

```
layer = iface.activeLayer()
if isinstance(layer,
QgsVectorLayer):
    print(layer.featureCount())
```

C++

```
auto *vec =
qobject_cast<QgsVectorLayer *>(
                iface-
>activeLayer());
if (vec) {
  qDebug() << vec->featureCount();
}
```

> **ყურადღება**
> `qobject_cast` აბრუნებს `nullptr`-ს, თუ გარდაქმნა შეუძლებელია, ანუ აქტიური ფენა რასტრულია ან საერთოდ არ არის არჩეული. ყოველთვის შეამოწმე შედეგი `nullptr`-ზე.

07.5 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაამატე `RasterLayer`-ს `bandCount()` და გამოიტანე `dynamic_cast`-ით.
2. შექმენი მესამე სახე `MeshLayer` და `type()` მიანიჭე `"mesh"`.
3. დაწერე ფუნქცია `int totalFeatures(const std::vector<Layer *> &layers)`, რომელიც მხოლოდ ვექტორული ფენების ობიექტებს დაითვლის.

შეჯამება

- მემკვიდრე კლასი იღებს ბაზისური კლასის ყველაფერს და ამატებს საკუთარს.
- `virtual` + `override` = პოლიმორფიზმი.
- ბაზისური ტიპიდან მემკვიდრეზე: `dynamic_cast` (QGIS-ში `qobject_cast`), შედეგი უნდა შემოწმდეს.

CHAPTER 08

08

მეხსიერების მართვა და smart pointers

Memory management and smart pointers

stack და heap · new/delete · memory leak · unique_ptr · shared_ptr · QGIS-ის მფლობელობის წესები

Python-ში მეხსიერებას ნაგვის შემგროვებელი (garbage collector) ასუფთავებს. C++-ში ეს შენი საქმეა. კარგი ამბავი ისაა, რომ თანამედროვე C++-ში ამას smart pointer-ები აკეთებენ.

08.1 stack და heap

	stack	heap
რა ინახება	ლოკალური ცვლადები	`new`-ით შექმნილი ობიექტები
ვინ ასუფთავებს	ავტომატურად, scope-ის დასრულებისას	პროგრამისტი (ან smart pointer)
სიჩქარე	ძალიან სწრაფი	ნელი
ზომა	შეზღუდული	დიდი

> **სასტუმრო და საკუთარი სახლი**
> stack სასტუმროა: ოთახი ავტომატურად გათავისუფლდება, როცა წახვალ. heap — საკუთარი ბინაა: შენ ვალდებული ხარ, დროზე დააბრუნო ან გაყიდო, თორემ სამუდამოდ დაკავებული დარჩება (memory leak).

08.2 new და delete

```C++
Polygon *p = new Polygon("Parcel-1");   // heap-ზე შეიქმნა
// ... ვიყენებთ ...
delete p;                               // ხელით უნდა წავშალოთ
```

თუ `delete` დაგავიწყდა, მეხსიერება გაჟონავს (leak). თუ ორჯერ წაშალე ან წაშლის შემდეგ გამოიყენე, პროგრამა დაიმსხვრევა. ხელით `new`/`delete` თანამედროვე კოდში თითქმის აღარ იწერება.

08.3 unique_ptr: ერთადერთი მფლობელი

`std::unique_ptr` ობიექტს ფლობს და scope-ის დასრულებისას ავტომატურად წაშლის.

C++ unique.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <utility>

struct Polygon {
    std::string name;
    explicit Polygon(std::string n) : name(std::move(n)) {
        std::cout << "created " << name << "\n";
    }
    ~Polygon() { std::cout << "destroyed " << name << "\n"; }
};

int main() {
    {
        auto p = std::make_unique<Polygon>("Parcel-1");
        std::cout << "using " << p->name << "\n";
    }
    std::cout << "after scope\n";
    return 0;
}
```

შედეგი

```
created Parcel-1
using Parcel-1
destroyed Parcel-1
after scope
```

`delete` არსად დაგვიწერია, მაგრამ ობიექტი ზუსტად ბლოკის ბოლოს განადგურდა. ამ პრინციპს RAII ჰქვია: რესურსი ობიექტის სიცოცხლეს ებმება.

08.4 მფლობელობის გადაცემა: std::move

`unique_ptr`-ის კოპირება შეუძლებელია, რადგან მფლობელი ერთია. შეიძლება მხოლოდ გადაცემა.

C++

```
auto a = std::make_unique<Polygon>("Parcel-2");
auto b = std::move(a);          // ახლა b ფლობს, a ცარიელია

if (a == nullptr) {
    std::cout << "a is empty\n";
}
```

შედეგი

```
created Parcel-2
a is empty
destroyed Parcel-2
```

ბოლო ხაზი პროგრამის დასრულებისას ჩნდება, როცა `b` იშლება.

> **QGIS-ში**
> `std::unique_ptr<QgsAbstractGeometry>` ხშირად გვხვდება: ფუნქცია, რომელიც ახალ გეომეტრიას ქმნის, მის მფლობელობას გვიბრუნებს. ჩანაწერი `std::unique_ptr<...>` ხელმოწერაში ნიშნავს „ეს შენია, შენ უნდა მართო“.

08.5 shared_ptr: ბევრი მფლობელი

როცა ერთ ობიექტს რამდენიმე ადგილი იყენებს, გამოიყენება `std::shared_ptr`. ობიექტი წაიშლება მაშინ, როცა ბოლო მფლობელიც გაქრება (reference counting).

C++ shared.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <utility>

struct Polygon {
    std::string name;
    explicit Polygon(std::string n) : name(std::move(n)) {
        std::cout << "created " << name << "\n";
    }
    ~Polygon() { std::cout << "destroyed " << name << "\n"; }
};

int main() {
    auto a = std::make_shared<Polygon>("Shared");
    std::cout << a.use_count() << "\n";
    {
        auto b = a;
        std::cout << a.use_count() << "\n";
    }
    std::cout << a.use_count() << "\n";
    return 0;
}
```

შედეგი

```
created Shared
1
2
1
destroyed Shared
```

08.6 რა ხდება QGIS-ში

QGIS-ს საკუთარი მფლობელობის წესები აქვს და დოკუმენტაცია მათ განმარტავს:

სიტუაცია	წესი
`QgsProject::addMapLayer(layer)`	პროექტი იღებს ფენის მფლობელობას და თვითონ წაშლის
Qt-ის `QObject` ხე	მშობელი შვილებს თავად წაშლის (იხ. თავი 10)
`std::unique_ptr<T>` დაბრუნება	მიმღები მფლობელია
მაჩვენებელი `T *` ხელმოწერაში	ჩვეულებრივ მფლობელობა არ გადადის

> **ყურადღება**
> თუ ფენას `new QgsVectorLayer(...)`-ით ქმნი და პროექტს აძლევ, თავად `delete` აღარ დაწერო. ფენა პროექტის საკუთრებაა და ორმაგი წაშლა პროგრამას დაამსხვრევს.

08.7 სავარჯიშო და შეჯამება

> **სავარჯიშო**
> 1. გადააკეთე თავი 04-ის `Layer rivers{...}` მაგალითი `std::make_unique<Layer>(...)`-ზე.
> 2. შექმენი `std::vector<std::unique_ptr<Polygon>>` და ჩაამატე სამი მრავალკუთხედი (`push_back(std::make_unique<...>(...))`). რა თანმიმდევრობით წაიშლებიან?
> 3. შეეცადე `unique_ptr`-ის კოპირებას (`auto b = a;`) და წაიკითხე შეცდომა.

შეჯამება

- stack ავტომატურია, heap — მფლობელის პასუხისმგებლობა.
- ხელით `new` / `delete` თავიდან აიცილე, გამოიყენე `make_unique` / `make_shared`.
- `unique_ptr` = ერთი მფლობელი, `shared_ptr` = ბევრი.
- QGIS-ში ყოველთვის გაარკვიე, ვინ ფლობს ობიექტს.

CHAPTER 09

09

STL: კონტეინერები და ალგორითმები

The Standard Template Library

vector · map · range-for · lambda · sort · count_if · accumulate

STL არის C++-ის სტანდარტული ბიბლიოთეკა კონტეინერებისა და ალგორითმებისთვის. QGIS ძირითადად Qt-ის კონტეინერებს იყენებს (თავი 10), მაგრამ STL-ის ცოდნა მაინც აუცილებელია: ორივეს ლოგიკა ერთია და ბევრი QGIS ფუნქცია STL ტიპებს აბრუნებს.

09.1 vector — დინამიური მასივი

`std::vector<T>` არის Python-ის `list`-ის ანალოგი, ოღონდ ყველა ელემენტი ერთი ტიპისაა.

PYTHON

```python
areas = [12.5, 7.0, 30.5, 4.0]
areas.append(9.0)
print(len(areas))
for a in areas:
    print(a)
```

C++

```cpp
std::vector<double> areas = {12.5,
7.0, 30.5, 4.0};
areas.push_back(9.0);
std::cout << areas.size() << "\n";
for (double a : areas) {
  std::cout << a << "\n";
}
```

ოპერაცია	Python	C++
დამატება	`append(x)`	`push_back(x)`
ზომა	`len(a)`	`a.size()`
ელემენტი	`a[0]`	`a[0]`
ბოლო	`a[-1]`	`a.back()`
გასუფთავება	`a.clear()`	`a.clear()`

> **ᲬᲔᲡᲘ**
> დიდი ელემენტების ციკლში გამოიყენე `const auto &`: `for (const auto &feature : features)`. ასე ელემენტი არ დაკოპირდება (თავი 05).

09.2 map — გასაღები → მნიშვნელობა

`std::map<K, V>` არის `dict`-ის ანალოგი. გასაღებებს ავტომატურად ალაგებს ზრდადობით.

C++ map.cpp

```cpp
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> population = {
        {"Tbilisi", 1200000},
        {"Batumi", 170000},
        {"Kutaisi", 147000},
    };

    population["Rustavi"] = 125000;

    for (const auto &[city, people] : population) {
        std::cout << city << ": " << people << "\n";
    }
    return 0;
}
```

შედეგი

```
Batumi: 170000
Kutaisi: 147000
Rustavi: 125000
Tbilisi: 1200000
```

შენიშვნა

მოსახლეობის რიცხვები აქ მიახლოებითი და საილუსტრაციოა, მიზანი კონტეინერის ჩვენებაა. `const auto &[city, people]` (structured binding, C++17) ერთდროულად ორივეს ამოიღებს.

ყურადღება

`population["Poti"]` — თუ გასაღები არ არსებობს, `map` ჩუმად შექმნის ახალ ჩანაწერს ნულოვანი მნიშვნელობით. არსებობის შესამოწმებლად გამოიყენე `population.find("Poti") != population.end()` ან `population.count("Poti")`.

09.3 ალგორითმები

`<algorithm>` და `<numeric>` გვაძლევს მზა ოპერაციებს: დალაგება, დათვლა, ჯამი.

C++ algo.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<double> areas = {12.5, 7.0, 30.5, 4.0};

    std::sort(areas.begin(), areas.end(),
              [](double a, double b) { return a > b; });

    for (double a : areas) std::cout << a << " ";
    std::cout << "\n";

    double total = std::accumulate(areas.begin(), areas.end(), 0.0);
    std::cout << "Total: " << total << "\n";
    std::cout << "Average: " << total / areas.size() << "\n";

    long big = std::count_if(areas.begin(), areas.end(),
                             [](double a) { return a > 10.0; });
    std::cout << "Parcels > 10 ha: " << big << "\n";
    return 0;
}
```

შედეგი

```
30.5 12.5 7 4
Total: 54
Average: 13.5
Parcels > 10 ha: 2
```

09.4 lambda — უსახელო ფუნქცია

`[](double a, double b) { return a > b; }` არის lambda: ფუნქცია, რომელიც ადგილზევე იწერება. Python-ში ეს `lambda a, b: a > b`-ს ჰგავს. `[]` კვადრატულ ფრჩხილებში შეიძლება გარე ცვლადების „დაჭერა“:

```cpp
double limit = 10.0;
auto isBig = [limit](double a) { return a > limit; };
```

QGIS-ში lambda-ებს ხშირად გამოიყენებენ signal-ებთან დასაკავშირებლად (თავი 10).

09.5 ძებნა

```cpp
auto it = std::find_if(areas.begin(), areas.end(),
                       [](double a) { return a > 20.0; });
if (it != areas.end()) {
    std::cout << "First > 20: " << *it << "\n";
}
```

`find_if` აბრუნებს **iterator-ს**: ელემენტზე მიმთითებელს (მაჩვენებლის მსგავს ობიექტს). ბოლოს `end()` ნიშნავს „ვერ ვიპოვე“. `*it` იღებს მნიშვნელობას. QGIS-ში `QgsFeatureIterator` ამავე იდეაზეა აგებული.

09.6 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაწერე `std::vector<Point>` ხუთი წერტილით და `std::sort`-ით დაალაგე x-ის მიხედვით.
2. შექმენი `std::map<std::string, double>`: მდინარე → სიგრძე კმ-ში (`{"Mtkvari", 1364}`, `{"Rioni", 327}`, `{"Alazani", 407}`). იპოვე ყველაზე გრძელი (`std::max_element` მიმართე დოკუმენტაციას).
3. `count_if`-ით დათვალე მდინარეები, რომლებიც 400 კმ-ზე გრძელია.

შეჯამება

- `vector` = list, `map` = dict, ორივე ტიპიზებულია.
- ალგორითმები იღებენ დიაპაზონს `begin()` – `end()` და lambda-ს.
- iterator — მაჩვენებლის მსგავსი ობიექტი კონტეინერში გადასაადგილებლად.

CHAPTER 10

10

Qt: QString, QVariant, signals და slots

Qt essentials for QGIS

QString · QList · QVariant · QgsAttributes · QObject-ის ხე · signal/slot · lambda connect

QGIS Qt-ზეა აგებული. API-ში `std::string`-ს თითქმის ვერ ნახავ: ყველგან `QString`, `QList`, `QVariant`. ამ თავში იმ მინიმუმს ვისწავლით, რომელიც QGIS-ის კოდის წასაკითხად საჭიროა.

10.1 QString

`QString` არის Unicode ტექსტი, ამიტომ ქართული ასოებიც პრობლემის გარეშე მუშაობს.

```
C++
QString name = QStringLiteral("მტკვარი");
QString info = QString("%1: %2 კმ").arg(name).arg(1364);

qDebug() << info.length();
```

ოპერაცია	Python	Qt
სიგრძე	`len(s)`	`s.length()`
ნაწილი	`s[2:5]`	`s.mid(2, 3)`
ჩანაცვლება	`s.replace(a, b)`	`s.replace(a, b)`
ფორმატირება	`f"{a}: {b}"`	`QString("%1: %2").arg(a).arg(b)`
რიცხვად	`int(s)`	`s.toInt()`, `s.toDouble()`
std::string-ში		`s.toStdString()`

> **ქართული და STD::STRING**
>
> `toStdString()` UTF-8-ს აბრუნებს. კონსოლში ქართული ტექსტი მხოლოდ მაშინ გამოჩნდება სწორად, თუ ტერმინალი UTF-8-ზეა. `qDebug()`-ით დაბეჭდვისას ტექსტი ბრჭყალებშია (`"..."`), `qDebug().noquote()` კი ბრჭყალებს გაუქმებს.

10.2 QList

`QList<T>` არის Qt-ის დინამიური მასივი, `std::vector`-ის ძმა.

```C++
QList<QString> cities = {"Tbilisi", "Batumi"};
cities.append("Kutaisi");
for (const QString &c : cities) {
    qDebug() << c;
}
```

გამოძახება `append`, `size()`, `[]`, `contains()`. QGIS-ში ფენების სია არის `QList<QgsMapLayer *>`, მაგალითად `QgsProject::mapLayersByName(...)` სწორედ ამას აბრუნებს.

10.3 QVariant — უნივერსალური ყუთი

`QVariant` ერთ მნიშვნელობას ინახავს ნებისმიერი ტიპისას: რიცხვი, ტექსტი, თარიღი, ან „არაფერი“ (NULL).

C++ variant.cpp

```cpp
#include <QString>
#include <QVariant>
#include <iostream>

int main() {
    QVariant population = 1200000;
    QVariant city = QString("Tbilisi");
    QVariant lat = 41.7151;
    QVariant unknown;                    // NULL

    std::cout << population.toInt() << "\n";
    std::cout << city.toString().toStdString() << "\n";
    std::cout << lat.toDouble() << "\n";
    std::cout << std::boolalpha << unknown.isNull() << "\n";
    return 0;
}
```

შედეგი

```
1200000
Tbilisi
41.7151
true
```

> **დაიმახსოვრე**
> `QVariant` არ „იცის“ წინასწარ, რა ტიპია შიგნით, ამიტომ კითხვისას თავად ვირჩევთ `toInt()`, `toString()`, `toDouble()`. თუ გარდაქმნა შეუძლებელია, შედეგი ნულოვანი ან ცარიელია.

10.4 QgsAttributes — ატრიბუტების ცხრილის ერთი სტრიქონი

ახლა გასაგებია `QgsAttributes` გვერდი, რომლითაც კურსი დაიწყო. ეს კლასი არის `QVariant`-ების მიმდევრობა: ფენის ცხრილის ერთი ჩანაწერი. თითოეული ელემენტი ერთი ველის მნიშვნელობაა.

C++ attrs.cpp

```cpp
#include <QList>
#include <QString>
#include <QVariant>
#include <iostream>

int main() {
    // ველები: name, length_km, comment
    QList<QVariant> attrs = {
        QVariant(QString("Mtkvari")),
        QVariant(1364),
        QVariant(),
    };

    for (const QVariant &v : attrs) {
        std::cout << (v.isNull() ? "NULL" : v.toString().toStdString()) <<
"\n";
    }
    return 0;
}
```

შედეგი

```
Mtkvari
1364
NULL
```

PYTHON

```python
feature = layer.getFeature(1)
print(feature.attributes())
# ['Mtkvari', 1364, NULL]
```

C++

```cpp
QgsFeature feature = layer->getFeature(1);
QgsAttributes attrs = feature.attributes();
QVariant name = attrs.at(0);
```

10.5 QObject-ის ხე

`QObject`-ები მშობელ-შვილის ხეში ცხოვრობენ. როცა მშობელი იშლება, თავის შვილებსაც წაშლის. ამიტომ Qt-ში ხშირად `new` გვხვდება `delete`-ის გარეშე.

C++

```cpp
auto *dialog = new QDialog();
auto *button = new QPushButton("OK", dialog);   // dialog არის მშობელი
// delete dialog;  ->  button-იც ავტომატურად წაიშლება
```

10.6 signals და slots

signal არის „რაღაც მოხდა“, slot — „ამაზე რეაგირება“. `connect` მათ აკავშირებს.

C++

```cpp
connect(QgsProject::instance(), &QgsProject::layersAdded,
        this, &MyTool::onLayersAdded);
```

ან lambda-თი, დამატებითი მეთოდის გარეშე:

C++

```cpp
connect(button, &QPushButton::clicked, this, [this]() {
    qDebug() << "clicked";
});
```

საფოსტო ყუთი

signal არის ზარის ხმა კარზე: ის არავის ეუბნება, ვინ გაიგონებს. slot არის ის, ვინც ზარზე რეაგირებს. `connect` ამათ ერთმანეთს აკავშირებს და გამომგზავნს მიმღები არ აინტერესებს.

PyQGIS-ში იგივეა: `layer.selectionChanged.connect(handler)`.

10.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. შექმენი `QList<QVariant>` ფენის ჩანაწერისთვის: სახელი (ტექსტი), მოსახლეობა (`int`), ფართობი (`double`). დაბეჭდე თითოეული ელემენტი.
2. შექმენი `QMap<QString, QVariant>` იგივე მონაცემით და გამოიტანე ველი სახელით.
3. წაიკითხე `QgsFeature::attribute` მეთოდი api.qgis.org-ზე და ჩამოწერე მისი ორი ვარიანტი.

შეჯამება

- `QString` = Unicode ტექსტი, `QList<T>` = მასივი, `QVariant` = უნივერსალური მნიშვნელობა.
- `QgsAttributes` = `QVariant`-ების მიმდევრობა.
- `QObject` ხე თავად ასუფთავებს შვილებს.
- signal/slot აკავშირებს „მოხდა“ და „რეაგირება“ ერთმანეთს.

CHAPTER 11

11

api.qgis.org-ის კითხვა

Reading the QGIS C++ API documentation

კლასის გვერდის სტრუქტურა · ხელმოწერების გაშიფვრა · სასარგებლო კლასები · სტრატეგია

თავიდან ეს გვერდები გაუგებარი იყო. ახლა უკვე გაქვს ყველა ინსტრუმენტი: ტიპები, მაჩვენებლები, მითითებები, კლასები, `QVariant`. ამ თავში სისტემატურად ვისწავლით დოკუმენტაციის „კითხვას“.

11.1 კლასის გვერდის სქემა

`api.qgis.org/api/<ვერსია>/class<სახელი>.html` გვერდს ერთი და იგივე სტრუქტურა აქვს:

ნაწილი	რას ეძებ იქ
Detailed Description	რისთვისაა კლასი, მაგალითები; ყოველთვის ჯერ ეს წაიკითხე
Inheritance diagram	ვისგან მოდის (იხ. თავი 07); მემკვიდრეობით მიღებული მეთოდები მშობლის გვერდზეა
Public Types	enum-ები და typedef-ები; აქ ჩანს, რას ნიშნავს კლასის სახელი
Public Member Functions	მეთოდები; ძირითადი სამუშაო
Static Public Member Functions	გამოიძახება კლასზე, ობიექტის გარეშე: `QgsProject::instance()`
Signals / Slots	რა მოვლენებს აგზავნის
Member Function Documentation	თითოეული მეთოდის დეტალები, `Since QGIS 3.x` ნიშნები

> **წესი**
> პირველად გვერდის ბოლოს ნუ იწყებ. წაიკითხე **Detailed Description**, შემდეგ მეთოდების სია, შემდეგ კონკრეტული მეთოდი.

11.2 ხელმოწერის გაშიფვრა

ავიღოთ რამდენიმე რეალური ჩანაწერი და წავიკითხოთ ისინი ისე, როგორც კომპილატორი კითხულობს.

ჩანაწერი	კლასი	როგორ იკითხება
`QgsGeometry geometry() const`	`QgsFeature`	ობიექტის გეომეტრია, ასლად; მეთოდი ობიექტს არ ცვლის
`void setGeometry(const QgsGeometry &geometry)`	`QgsFeature`	გეომეტრიას ვუთითებ; კოპირება არ ხდება, ორიგინალს არ შეცვლის
`QVariant attribute(const QString &name) const`	`QgsFeature`	ველის მნიშვნელობა სახელით; `QVariant`-ად
`QgsFeatureId id() const`	`QgsFeature`	ობიექტის უნიკალური ნომერი
`bool isValid() const`	`QgsFeature`	ობიექტი ვალიდურია თუ არა
`QgsFeatureIterator getFeatures(const QgsFeatureRequest &request = QgsFeatureRequest()) const`	`QgsVectorLayer`	ობიექტების iterator; პარამეტრი არასავალდებულოა (default)
`double area() const`	`QgsGeometry`	გეომეტრიის ფართობი
`static QgsGeometry fromPointXY(const QgsPointXY &point)`	`QgsGeometry`	`static`: გამოიძახება `QgsGeometry::fromPointXY(...)`-ით
`QList<QgsMapLayer *> mapLayersByName(const QString &name) const`	`QgsProject`	ფენების სია სახელით; ელემენტები მაჩვენებლებია, შეამოწმე ცარიელია თუ არა

11.3 მაკროები, რომლებიც შეგხვდება

ჩანაწერი	აზრი
`SIP_SKIP`, `SIP_OUT`, `SIP_FACTORY`	მითითებები Python-ის დამაკავშირებელი გენერატორისთვის (SIP); C++-ში შეგიძლია უგულებელყო
`Q_INVOKABLE`	მეთოდი Qt-ის მეტა-სისტემიდან გამოიძახება
`Q_DECL_DEPRECATED`	მოძველებულია, ნუ გამოიყენებ
`CORE_EXPORT`	კლასი ბიბლიოთეკიდან გამოდის; ტექნიკური დეტალია
`Since QGIS 3.xx`	მეთოდი ამ ვერსიიდან არსებობს

11.4 ცოცხალი მაგალითი: ამოცანიდან მეთოდამდე

ამოცანა: ფენაში თითოეული ობიექტის ფართობი გამოვთვალოთ.

1. იპოვე ფენის კლასი: `QgsVectorLayer`.
2. იქ ნახე `getFeatures()` → აბრუნებს `QgsFeatureIterator`-ს.
3. იტერატორიდან მიიღებ `QgsFeature`-ს.
4. `QgsFeature::geometry()` → `QgsGeometry`.
5. `QgsGeometry::area()` → `double`.

C++

```
QgsFeatureIterator it = layer->getFeatures();
QgsFeature feature;
while (it.nextFeature(feature)) {
    double area = feature.geometry().area();
    qDebug() << feature.id() << area;
}
```

PYTHON

```
for f in layer.getFeatures():
    print(f.id(),
f.geometry().area())
```

C++

```
QgsFeature f;
QgsFeatureIterator it = layer-
>getFeatures();
while (it.nextFeature(f)) {
  qDebug() << f.id() <<
f.geometry().area();
}
```

შენიშვნა

`area()` ფენის CRS ერთეულებში აბრუნებს ფართობს (გეოგრაფიულ CRS-ში ეს გრადუსია). მეტრებში ზუსტი გამოთვლისთვის `QgsDistanceArea` გამოიყენება. ასეთი ნიუანსები ხშირად **Detailed Description**-შია.

11.5 ძირითადი კლასები ერთ ცხრილში

კლასი	რას აკეთებს	PyQGIS-ის ანალოგი
`QgsProject`	პროექტი, ფენების მფლობელი	`QgsProject.instance()`
`QgsVectorLayer`	ვექტორული ფენა	იგივე
`QgsRasterLayer`	რასტრული ფენა	იგივე
`QgsFeature`	ობიექტი: ID, გეომეტრია, ატრიბუტები	იგივე
`QgsGeometry`	გეომეტრია და ოპერაციები	იგივე
`QgsPointXY`	2D წერტილი	იგივე
`QgsCoordinateReferenceSystem`	CRS	იგივე
`QgsCoordinateTransform`	გადაყვანა CRS-ებს შორის	იგივე
`QgsFeatureRequest`	ფილტრი ობიექტების არჩევისას	იგივე

11.6 შეცდომის კითხვა

კომპილატორის შეცდომა პირველად საშიშად გამოიყურება, მაგრამ ხშირად ერთი ხაზი გვეუბნება მთავარს:

შეცდომა	ჩვეულებრივი მიზეზი
`'QgsFeature' was not declared`	ჩასართავი `#include <qgsfeature.h>` გაგრჩა
`undefined reference to ...`	ბიბლიოთეკა არ არის მიბმული (`target_link_libraries`)
`passing 'const X' as 'this' discards qualifiers`	`const` ობიექტზე არა-`const` მეთოდი გამოიძახე
`cannot convert 'QString' to 'std::string'`	გამოიყენე `.toStdString()`

11.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. გახსენი `QgsAttributes` გვერდი. იპოვე მშობელი კლასი და ჩამოწერე სამი მეთოდი.
2. იპოვე `QgsGeometry::buffer` ხელმოწერა. რა პარამეტრები აქვს? რას აბრუნებს? რომელი პარამეტრია `const`?
3. იპოვე, რა მეთოდი უნდა გამოიყენო ფენაში არჩეული ობიექტების დასათვლელად (`QgsVectorLayer`-ის გვერდზე ეძებე „selected“).
4. გადააკეთე წინა მაგალითი ისე, რომ მხოლოდ ისეთი ობიექტები დაითვალოს, რომელთა ფართობი 1000-ზე მეტია.

შეჯამება

- ჯერ **Detailed Description**, მერე მეთოდების სია.
- ხელმოწერა იკითხება მარცხნიდან: დაბრუნების ტიპი, სახელი, პარამეტრები (`*` / `&` / `const`), მეთოდის `const`.
- PyQGIS-ის ცოდნა = დოკუმენტაციის ცოდნა: სახელები ერთია.

CHAPTER 12

12

პროექტი: საკუთარი ინსტრუმენტი QGIS-ში

Capstone — your own tool inside QGIS

QGIS-ის აწყობა წყაროდან · დამოუკიდებელი C++ აპლიკაცია · C++ პლაგინის ჩონჩხი · CMake · ლიცენზია

საბოლოო მიზანი: საკუთარი C++ კოდი QGIS-თან ერთად გაუშვა. გზა სამ საფეხურად ვაგებთ: ჯერ ვმუშაობთ QGIS-ის ბიბლიოთეკასთან დამოუკიდებელ აპლიკაციაში, შემდეგ ვაკეთებთ პლაგინს, ბოლოს ვაწყობთ QGIS-ს წყაროდან.

12.1 გეგმა

საფეხური	შედეგი	სირთულე
1	კონსოლური პროგრამა, რომელიც `qgis_core`-ს იყენებს	დაბალი
2	C++ პლაგინი, რომელიც QGIS-ის მენიუში ჩნდება	საშუალო
3	QGIS-ის აწყობა წყარო კოდიდან (შენი კოდით ან მის გარეშე)	მაღალი

რჩევა

ბევრი ადამიანი პირდაპირ მესამე საფეხურზე ხტება და იჭედება აწყობის დეტალებში. პირველი ორი უკვე გასაგებ შედეგს იძლევა და გასწავლის ბიბლიოთეკების ერთად მუშაობას.

12.2 საფეხური 1: პირველი პროგრამა qgis_core-ზე

ამოცანა: ფენის ობიექტების დათვლა და ფართობების ჯამი.

C++ main.cpp

```cpp
#include <iostream>

#include <qgsapplication.h>
#include <qgsfeature.h>
#include <qgsgeometry.h>
#include <qgsvectorlayer.h>

int main(int argc, char *argv[]) {
    QgsApplication app(argc, argv, false);     // false = GUI არ გვჭირდება
    QgsApplication::initQgis();

    QgsVectorLayer layer("rivers.geojson", "rivers", "ogr");
    if (!layer.isValid()) {
        std::cerr << "Layer failed to load\n";
        return 1;
    }

    std::cout << "Features: " << layer.featureCount() << "\n";

    double totalArea = 0.0;
    QgsFeature feature;
    QgsFeatureIterator it = layer.getFeatures();
    while (it.nextFeature(feature)) {
        totalArea += feature.geometry().area();
    }
    std::cout << "Total area: " << totalArea << "\n";

    QgsApplication::exitQgis();
    return 0;
}
```

შედეგი

```
Features: 42
Total area: 0.183762
```

რიცხვები საილუსტრაციოა: ისინი შენი მონაცემის მიხედვით განსხვავდება. ყურადღება მიაქციე `layer.isValid()` შემოწმებას: ფენის ჩატვირთვა შეიძლება ჩაიშალოს.

12.3 CMake

`CMakeLists.txt` უთხრის კომპილატორს, სად არის QGIS-ის header-ები და ბიბლიოთეკები.

CMAKE CMakeLists.txt

```cmake
cmake_minimum_required(VERSION 3.16)
project(feature_counter LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)

# QGIS 4 = Qt6, QGIS 3 = Qt5. აირჩიე შენი QGIS-ის შესაბამისი.
find_package(Qt6 REQUIRED COMPONENTS Core Gui Xml Widgets)

find_path(QGIS_INCLUDE_DIR qgsapplication.h PATH_SUFFIXES qgis)
find_library(QGIS_CORE_LIBRARY qgis_core)

add_executable(feature_counter main.cpp)
target_include_directories(feature_counter PRIVATE ${QGIS_INCLUDE_DIR})
target_link_libraries(feature_counter PRIVATE
    ${QGIS_CORE_LIBRARY} Qt6::Core Qt6::Gui Qt6::Xml Qt6::Widgets)
```

SHELL

```
cmake -S . -B build
cmake --build build
./build/feature_counter
```

> **გაითვალისწინე**
> `qgis_core` და Qt-ის ვერსიები უნდა ემთხვეოდეს იმას, რითაც თავად QGIS არის აწყობილი. პაკეტები (`qgis-dev` Linux-ზე, OSGeo4W Windows-ზე) ამ ჯგუფს ერთად გაძლევს. ზუსტი სახელები და ბილიკები სისტემაზეა დამოკიდებული.

12.4 საფეხური 2: C++ პლაგინის ჩონჩხი

C++ პლაგინი არის დინამიური ბიბლიოთეკა, რომელსაც QGIS გაშვებისას ტვირთავს. მას აქვს `QgisPlugin` ინტერფეისი და ექსპორტირებული ფუნქციები.

C++ mytool.h

```cpp
#pragma once
#include <QAction>
#include <QObject>
#include <qgisplugin.h>

class QgisInterface;

class MyTool : public QObject, public QgisPlugin {
    Q_OBJECT
public:
    explicit MyTool(QgisInterface *iface);

    void initGui() override;       // QGIS გვიძახებს ჩატვირთვისას
    void unload() override;        // და გამორთვისას

private slots:
    void run();

private:
    QgisInterface *mIface = nullptr;
    QAction *mAction = nullptr;
};
```

C++ mytool.cpp

```cpp
#include "mytool.h"

#include <QMessageBox>
#include <qgisinterface.h>
#include <qgsvectorlayer.h>

MyTool::MyTool(QgisInterface *iface)
    : QgisPlugin(QStringLiteral("Feature Counter"),
                 QStringLiteral("Counts features of the active layer"),
                 QStringLiteral("Analysis"), QStringLiteral("0.1"),
                 QgisPlugin::UI),
      mIface(iface) {}

void MyTool::initGui() {
    mAction = new QAction(tr("Count features"), this);
    connect(mAction, &QAction::triggered, this, &MyTool::run);
    mIface->addPluginToMenu(tr("&My Tools"), mAction);
}

void MyTool::unload() {
    mIface->removePluginMenu(tr("&My Tools"), mAction);
    delete mAction;
    mAction = nullptr;
}

void MyTool::run() {
    auto *layer = qobject_cast<QgsVectorLayer *>(mIface->activeLayer());
    if (!layer) {
        QMessageBox::warning(nullptr, tr("Error"), tr("Select a vector
layer."));
        return;
    }
    QMessageBox::information(nullptr, tr("Result"),
```

```
        tr("Features: %1").arg(layer->featureCount()));
}
```

გამოიყენე ყველაფერი, რაც ისწავლე: `qobject_cast` + `nullptr` შემოწმება (თავები 04, 07), `connect` (თავი 10), `const` და `QString` (თავები 05, 10).

> **ჩონჩხი და რეალური კოდი**
> ზემოთ მოცემული კოდი არის სტრუქტურის ილუსტრაცია. პლაგინის ექსპორტირებული ფუნქციები (`classFactory`, `name`, `version` და სხვა) და `QgisPlugin`-ის კონსტრუქტორის ზუსტი ხელმოწერა ვერსიიდან ვერსიამდე შეიძლება შეიცვალოს. შენი QGIS-ის ვერსიის წყარო კოდში `src/plugins/` საქაღალდეშია მოქმედი მაგალითები. დაიწყე მათი კოპირებით და მერე შეცვალე.

12.5 საფეხური 3: QGIS-ის აწყობა წყაროდან

SHELL

```
git clone https://github.com/qgis/QGIS.git
cd QGIS
mkdir build && cd build
cmake -G Ninja -DCMAKE_BUILD_TYPE=Debug ..
ninja
```

ეტაპი	რა ხდება
`cmake`	ამოწმებს დამოკიდებულებებს (Qt, GDAL, GEOS, PROJ...) და ქმნის build ფაილებს
`ninja`	აკომპილებს; პირველად დიდ დროს მოითხოვს
ტესტი	გაუშვი აწყობილი `qgis`; შემდეგ დაამატე შენი პლაგინი `src/plugins/` ქვეშ

მცირე ცვლილების შეტანის შემდეგ `ninja` მხოლოდ შეცვლილ ფაილებს აკომპილებს.

ლიცენზია

QGIS ვრცელდება GNU GPL ლიცენზიით. თუ შენ მიერ შეცვლილ QGIS-ს ან მასში ჩაშენებულ კოდს სხვას გადასცემ, მიმღებს უფლება აქვს მოითხოვოს **წყარო კოდიც**. ამიტომ „დახურული ბინარული“ მხოლოდ საკუთარი გამოყენებისთვის მუშაობს. ბინარულ ფაილშიც კოდის აღდგენა სრულიად შეუძლებელი არ არის. საკუთარი ალგორითმის დასაცავად გააზრებულად აირჩიე არქიტექტურა და გადაამოწმე ლიცენზიის პირობები გავრცელებამდე.

12.6 სავარჯიშო და შეჯამება

საბოლოო დავალება

1. გააკეთე საფეხური 1 შენი მონაცემით (მაგ. რაიონების პოლიგონები) და გამოიტანე თითოეული რაიონის ფართობი.
2. დაამატე `QgsFeatureRequest` ფილტრი და დათვალე მხოლოდ ობიექტები გარკვეული ატრიბუტით.
3. წაიკითხე `src/plugins/` -ში არსებული ერთი მარტივი პლაგინი და შეადარე შენს ჩონჩხს.
4. მოამზადე 5-ხაზიანი დოკუმენტი: რა ისწავლე, რა გაუგებარია, რა გინდა შემდეგ ეტაპზე.

კურსის შეჯამება

- PyQGIS და C++ API ერთი სისტემაა; სახელები იგივეა, გრამატიკა — განსხვავებული.
- `*` — შეიძლება `nullptr`, `&` — აუცილებლად არსებობს, `const &` — უსაფრთხო წაკითხვა.
- smart pointer-ები და QObject-ის ხე მეხსიერებას თავად მართავენ.
- `api.qgis.org` იკითხება: აღწერა, ტიპები, ხელმოწერა.
- გზა: დამოუკიდებელი აპლიკაცია → პლაგინი → QGIS-ის აწყობა.